CAP Theorem
Back of the Envelope Calculation (Instagram)
Monolith vs Microservice Architecture

CAP Theorem

Scalability : Distributed System
Centralized (wrong)

CAP
C : Consistency : client gets the same data
at the same time no matter the node (db)

A : Availability : Whenever client sends a req, he should
get a response.

P : Partition Tolerance : our Application should always be Up
Always operate.

CA
CP
AP

CAP : No matter what, you will always get
max 2 out of these 3.

Parition Tolerance : This is Mandatory

Db1 and DB2 and db3 : Write operations not supported

Consistency Vs Avalability

Consistency : Reponse (Up to date mandatory)

Avalability : Reponse (up to date optional)
(Always get clear response is mandatory)

AVAIABILITY NUMBERS:

100% AVAILABE
365 DAYS

99% OF 365 DAYS : 3.65 DAYS DOWN
99.9% OF 365 DAYS : 8.77 HOURS
99.99%
99.999%
100% : IDEAL
(GOOGLE, AMAZON, MICROSOFT)

NUMBER OF 9S.

BACK OF THE ENVELOPE
CALCULATIONS

QPS : QUERY PER SECOND
STORAGE UNIT : 5 YEARS

READ : SELECT
WRITE :
INSERT
DELETE
UPDATE

INSTAGRAM
BACK OF THE ENVELOPE CALC:

POWER OF 2:
10 : THOUSANDS : KILOBYTE
20 : MILLIONS : MEGABYTE
30 : BILLIONS : GEGABYTE
40 : TRILLIONS : TERABYTE
50 : QUADRILLIONS : PETABYTE
60 : : EXABYTE

ASSUMPTIONS:
1. MONTHLY ACTICE USERS : ~2 BILLION
60% OF THESE USERS USE INSTA DAILY : ~1.2 BILLION
DAU : DAILY ACTIVE USERS : 1.2B

USER SEE FEED (1 FEED REQUEST -- 1 Q)
USER POST PHOTO/VIDEO/REEL (1 POST REQUEST -- 1 Q)

USER CHECK FEED 30 TIMES A DAY (AVERAGE)
ESTIMATE OF 5 YEARS.
USER PUBLISH ONE PICTURE/REEL A DAY. (AVERAGE)

FEED VIEW OPS:
OPS : DAILY FEED REQUEST : 1.2B*30/86,400 = ~420K/sec
PEEK OPS : 2 * OPS = ~840K/SEC

UPLOAD OPS:
OPS : DAILY UPLOAD REQ = 1.2B*1 / 86,400 = ~14K/SEC
PEAK OPS : 2 * OPS = ~28K/SEC

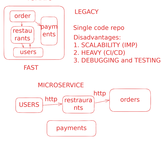
STORAGE UNIT:
ASSUMPTIONS:
20% OF USERS : VIDEO (50MB)
80% OF USERS : PHOTO (1 MB)

PHOTOS:
photostday = 1.2B*80% = ~1B
1Billion * 1 MB = 2^30 * 1 * 2^20 = 2^50 = ~1PB
Videos:
1.2B*20% = ~0.25B * 50MB = 0.25 * 2^30 * 50 * 2^20
2^50 * 12 = ~12PB

Total : 12PB + 1PB = 13 PB

calc for 5 years:
13PB/day * 365 days * 5 years = ~24000PB
24 * 10^3 = 24 * 2^10 * 2^50 = 24 * 2^60 = ~24 EB
(24 Exabyte)

MONOLITH and MICROSERVICE!



disadvantage:
1. SLOW Comparartively (Latency)
2. Transaction Management
3. Multiple microservice join.

nw issue
u1 -> u2

ACID : Atomocity
Transaction ROLLBACK

Different phases of a microservice:
1. Decomposition : (monolith -> microservice) ?? (complete)
2. Database (Shared DB or Unique DB)
3. Communication (API / Event driven system)

DevOps:
4. Deployment (How will your app will be deployed) CI/CD
5. Observability How will we monitor our application

Decomposition Phase:

If you have a monolith -> how will you break it to a
microservice architecture

Design patterns of Decomposition:
Decomposition by Business logic
decomposition by sub domains.

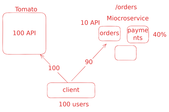
Disadvantage:
Developer should
be aware of
entire business

subdomain of orders
www.zomato.com/orders/pay
www.zomato.com/paymentds/

/orders. --> (1 microservice)
/payments --> (1 microservice)

/add/user

STRANGLER PATTERN
How will you do it???



DB Phase:

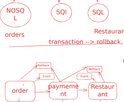
Each microservice can have

1. shared DB
2. Unique Db

Advantage:
Simple to carry operations
JOIN (SQL)
Transactions manage (ACID)

Disadvantage:
Can not be scaled properly
has the limitation of either
being sql or no SQL

User create an order



Disadvantage:
JOIN (CQRS)
Transaction Mgmt (SAGA)

transaction -> rollback, success

Choreography
Complex to make

Listeners

Event Driven System

Orchestration:

CQRS (COMMAND QUERY Request Sepration)
COMMAND : write opertation (Update, delete, insert)
QUERY : read (SELECT)

MASTER_SLAVE MODEL

READ (JOIN)